



Chewing on the AWS Twitter Feed



Dave Cuthbert

Jan 9 · 7 min read

Twitter streams tons of messages every minute. Have you ever wanted to work with the data and do your own analysis of the Twitter feed? A little while ago Assaf Mentzer put together a demonstration project to show off some AWS capabilities, ['Building a Near Real-Time Discovery Platform with AWS'](#), that offers a great introduction to working with streaming data.

Since then there have been a few changes in some of the AWS services he used. This page updates Assaf's instructions and adds a few additional tips for anyone getting started with the project. The steps on this page are fairly complete, but they are definitely an extension of Assaf's work. I hope these notes others to succeed with the Twitter feed.

The notes will make more sense if you follow along with Assaf's page open in another browser tab. Without further ado, here they are:

Prerequisites

No real changes here. The only note is to consider creating a new Twitter account for this project instead of reusing your usual one (if you have one). This way with these data explorations won't interfere with your usual settings and preferences.

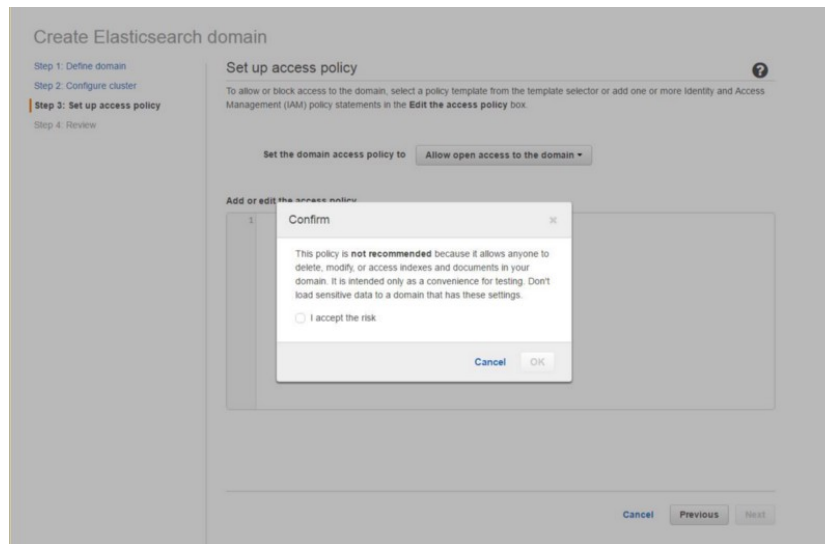
Create Amazon Elasticsearch Service cluster

- Sign in to the Elastic Search Service Console.
- If this is your first time choose 'Get Started', otherwise choose 'Create a New Domain'.

The screenshot shows the 'Create Elasticsearch domain' console page. On the left, a sidebar lists the steps: 'Step 1: Define domain' (highlighted), 'Step 2: Configure cluster', 'Step 3: Set up access policy', and 'Step 4: Review'. The main content area is titled 'Define domain' and includes a sub-header 'A domain is a collection of all the resources needed to run your Elasticsearch cluster.' Below this, there are two sections: 'Domain Name' and 'Version'. The 'Domain Name' section has a text input field with 'myesdomain' entered. A tooltip explains that the name must start with a lowercase letter and be between 3 and 28 characters, with valid characters being a-z, 0-9, and hyphen. The 'Version' section has a dropdown menu for 'Elasticsearch version' set to '5.1'. At the bottom right, there are 'Cancel' and 'Next' buttons.

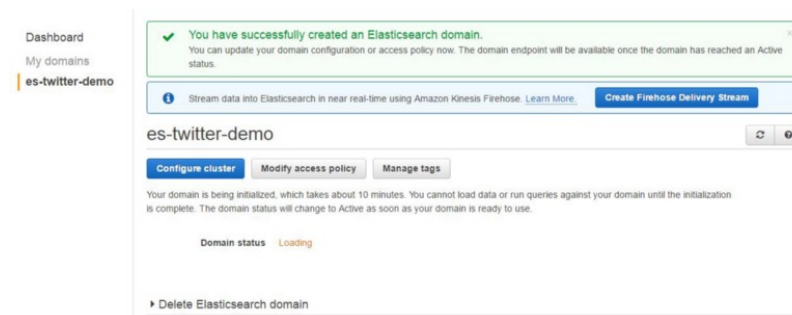
Create an Elasticsearch Domain

- Name your domain, we will use “es-twitter-demo” here, then choose ‘Next’.
- After naming the domain for your elasticsearch service Assaf recommends accepting the defaults on the next page. The defaults are reasonable, but it is cheaper to use a smaller EC instance to host the elasticsearch domain. The t2.medium.elasticsearch instance type works well for simple experimentation.
- Choose ‘Allow Open Access’ as the domain policy. NOTE: This is a poor security practice and the console will complain about it.



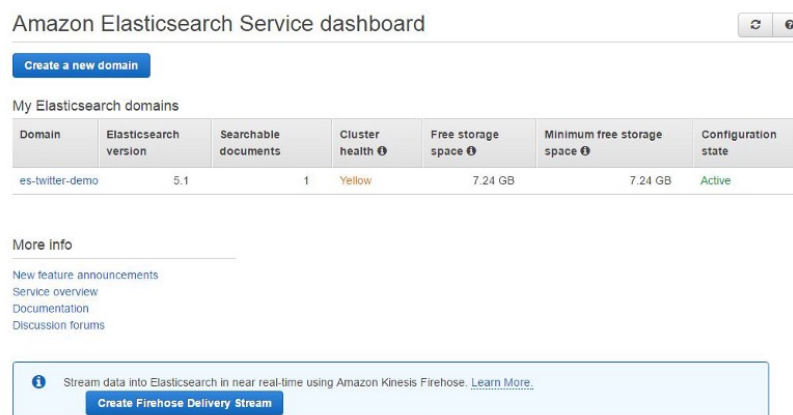
Set up an access policy

- Choose 'Confirm and Create'. It takes about 10 minutes for the domain to be set up.



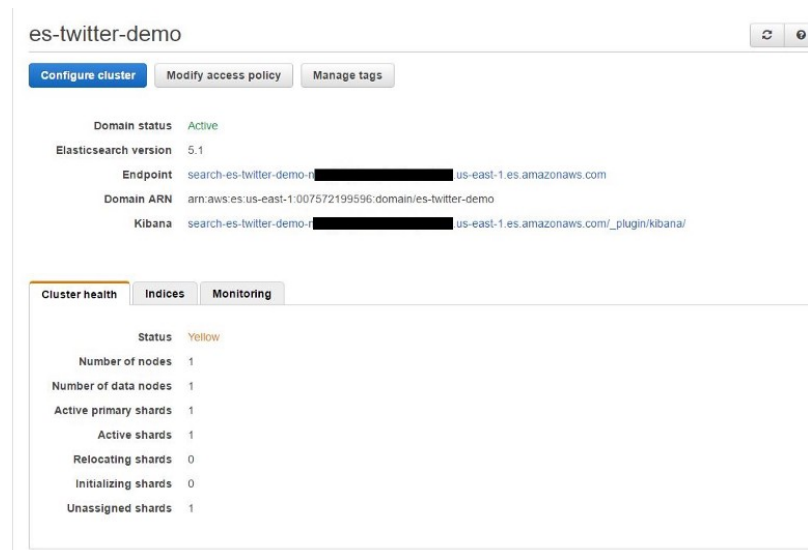
Domain creation confirmation dialog

- When the endpoint is active, click on it.



Elasticsearch Service Dashboard

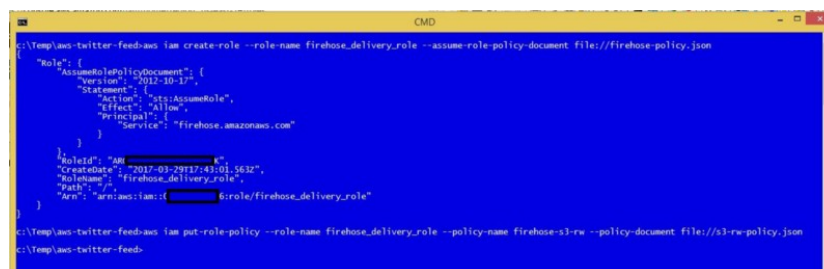
- The service console will display various details about the newly created service endpoint. Note this information down for later, especially the Elasticsearch service endpoint and the Kibana URL.



Configuration details

Create an IAM role for Firehose

- Create the two policy files that will be uploaded using the AWS command line client. NOTE: On my system, cutting and pasting these lines from the original article introduced various hidden artifacts in the files that caused the upload to fail. If that happens on your system, just retype them. The syntax Assaf gives in the original article is correct.
- Edit the s3-rw-policy.json file to use your S3 bucket



Edit the .json file

- Use the AWS CLI client to upload the files.
- Check that the policy has been uploaded properly



Verify access policy

Create a Lambda function

This section deviates from the original quite a bit. The outcome is the same, but the Lambda function setup and configuration process appears to have been updated since the original article was published.

- Download the deployment package and unzip to the s3-twitter-to-es-python folder.
- Modify the s3-twitter-to-es-python/config.py file. Edit the file so that the value of es_host matches Elasticsearch Service endpoint for your domain.

```

1  """
2  Created on Oct 12, 2015
3
4  @author: mantzxx
5  """
6
7  es_host = 'search-es-twitter-demo-XXXXXXXXXX.us-east-1.es.amazonaws.com'
8  es_port = 80
9  es_bulk_chunk_size = 1000 #number of documents to index in a single bulk operation
10
11

```

Edit host value

- Zip the folder content on your local environment as my-s3-twitter-to-es-python.zip NOTE: It is important to zip the entire folder contents not just zip up the folder itself.
- Sign in to the Lambda console.
- Choose 'Create a Lambda function'. NOTE: Choose 'Get started now' if this is your first time using Lambda.
- Choose 'Configure triggers' from the list of choices at top left of the screen

Lambda > New function

- Select blueprint
- Configure triggers**
- Configure function
- Review

Select blueprint

Blueprints are sample configurations of event sources and Lambda functions. Choose a blueprint that best aligns with your desired scenario and customize as needed, or skip this step if you want to author a Lambda function and configure an event source separately. Except where otherwise noted, blueprints are licensed under CC0.

Welcome to AWS Lambda! You can get started on creating your first Lambda function by choosing one of the blueprints below.

Select runtime Filter << Viewing 1-9 of 84 >>

Blank Function Configure your function from scratch. Define the trigger and deploy your code by stepping through our wizard. custom	kinesis-firehose-syslog-to-json An Amazon Kinesis Firehose stream processor that converts input records from RFC3164 Syslog format to JSON. nodejs - kinesis-firehose	alexa-skill-kit-sdk-factskill Demonstrate a basic fact skill built with the ASK NodeJS SDK. nodejs - alexa
batch-get-job-python27 Returns the current status of an AWS Batch Job. python2.7 - batch	kinesis-firehose-apache-log-to-j... An Amazon Kinesis Firehose stream processor that converts input records from Apache Common Log format to JSON. python2.7 - kinesis-firehose	cloudfront-modify-response-he... Blueprint for modifying CloudFront response header implemented in NodeJS. nodejs - cloudfront - response hea...
s3-get-object-python An Amazon S3 trigger that retrieves metadata for the object that has been updated. python2.7 - s3	config-rule-change-triggered An AWS Config rule that is triggered by configuration changes to EC2 instances. Checks instance types. nodejs4.3 - config	lex-book-trip-python Book details of a visit, using Amazon Lex to perform natural language understanding. python2.7 - lex

Cancel

Configure lambda triggers

- Click inside the dotted lines and choose the S3 source from the drop down list.

Lambda > New function

- Select blueprint
- Configure triggers**
- Configure function
- Review

Configure triggers

You can choose to add a trigger that will invoke your function.

 >  Lambda Remove

Filter integrations

- CloudWatch Events - Schedule
- CloudWatch Logs
- CodeCommit
- Cognito Sync Trigger
- DynamoDB
- Kinesis
- S3**
- SNS

Cancel Previous Next

Select a storage

- Enter your bucket name and be sure the 'Enable trigger' box is checked.

Lambda > New function

Select blueprint

Configure triggers

Configure function

Review

Configure triggers

You can choose to add a trigger that will invoke your function.

S3

Lambda

Remove

Bucket:

Event type:

Prefix:

Suffix:

Lambda will add the necessary permissions for Amazon S3 to invoke your Lambda function from this bucket. [Learn more about the Lambda permissions model.](#)

Enable trigger ☒

Cancel Previous Next

Name the S3 bucket

- On the next screen update the variables as follows:

Name: 's3-twitter-to-es-python'
Runtime: 'Python2.7'

Code entry: 'Upload a .ZIP file'
(Click the button to upload the .zip created a few steps ago.)

Handler: 'lambda_function.lambda_handler'

Role: 'Create new role from templates(s)'

Role name: lambda_s3_exec_role'

Memory: 128

Timeout: 2 minute

Lambda > New function

Select blueprint

Configure triggers

Configure function

Review

Configure function

A Lambda function consists of the custom code you want to execute. [Learn more about Lambda functions.](#)

Name*

Description

Runtime*

Lambda function code

Provide the code for your function. Use the editor if your code does not require custom libraries (other than boto3). If you need custom libraries, you can upload your code and libraries as a ZIP file.

Code entry type

Function package*
For files larger than 10 MB, consider uploading via S3.

You can define Environment Variables as key-value pairs that are accessible from your function code. These are useful to store configuration settings without the need to change function code. [Learn more.](#) For storing sensitive information, we recommend encrypting values using KMS and the console's encryption helpers.

Enable encryption helpers ☐

Environment variables

Key	Value

Lambda function handler and role

Lambda function configuration

Lambda function handler and role

Handler* ⓘ

Role* ⓘ
Lambda will automatically create a role with permissions from the selected policy templates. Note that basic Lambda permissions (logging to CloudWatch) will automatically be added. If your function accesses a VPC, the required permissions will also be added.

Role name* ⓘ

Policy templates ⓘ

▼ **Advanced settings**

These settings allow you to control the code execution performance and costs for your Lambda function. Changing your resource settings (by selecting memory) or changing the timeout may impact your function cost. [Learn more about how Lambda pricing works.](#)

Memory (MB)* ⓘ

Timeout* min sec

AWS Lambda will automatically retry failed executions for asynchronous invocations. You can additionally optionally configure Lambda to forward payloads that were not processed to a dead-letter queue (DLQ), such as an SQS queue or an SNS topic. [Learn more about Lambda's retry policy and DLQs.](#) **Please ensure your role has appropriate permissions to access the DLQ resource.**

DLQ Resource ⓘ

All AWS Lambda functions run securely inside a default system-managed VPC. However, you can optionally configure Lambda to access resources, such as databases, within your custom VPC. [Learn more about accessing VPCs within Lambda.](#) **Please ensure your role has appropriate permissions to configure VPC.**

VPC ⓘ

Environment variables are encrypted at rest using a default Lambda service key. You can change the key below to one of your account's keys or paste in a full KMS key ARN.

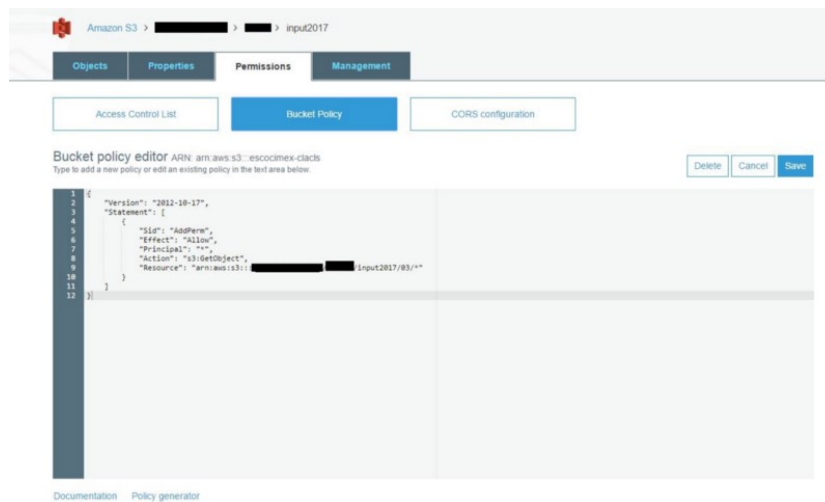
Additional configuration

- Create the function
- Verify the role was created properly



Confirm configuration

- Verify the S3 bucket has permissions set properly



Check S3 bucket permissions

Feed the producer with Twitter streaming data

That's a lot of configuration and backend set up so far, but we are nearly there. The next steps are where the fun begins. Assaf configured a fairly restrictive data feed. You can change his configuration to select tweets which you think are more interesting.

Using the suggested t2.micro instance for the node.js host works well for the demo. Be aware that the stock configuration file is limited to US data and it only sends a few of the available fields from the Twitter stream. If you really want to pull in lots of data you may want to upgrade your server instances. That said, when starting out it is probably best to stick with the standard configuration.

If you want to change the data streams later on, here are some tips on how to do it.

Add countries outside the US to the stream

This sends a LOT of data for the demo to process.

- Go to the node.js host.
- Change directory to twitter-streaming-firehose-nodejs.
- Edit config.js to comment out the regional filter.
- Restart the node server.

```
function _sendToFirehose() {
  var locations = [ '-180,-90,180,90' ]; //all the world
  var stream = T.stream('statuses/filter', { locations: config.locations });

  var records = [];
  var record = {};
  var recordParams = {};
  stream.on('tweet', function (tweet) {
    //if (tweet.coordinates){
    //  if (tweet.coordinates !== null){
    console.log(JSON.stringify(tweet));
    recordParams = {
      DeliveryStreamName: config.firehose.DeliveryStreamName,
      Record: {
        Data: JSON.stringify(tweet)+'\n'
      }
    };
    firehose.putRecord(recordParams, function(err, data) {
      if (err){
        log.error(err);
      }
    });
  });
  // }
} // }
```

Edit config.js

Add new fields to the stream

That was a great start, but you are probably anxious to gather other types of meta data. You can increase the amount of data collected in each Tweet by modifying the Lambda Function. Here's how:

- Go to the s3-twitter-to-es-python directory.
- Edit tweet_utils.py to add additional fields in the get_tweets() function.
- Rob Johnson has compiled a list of [available fields](#).

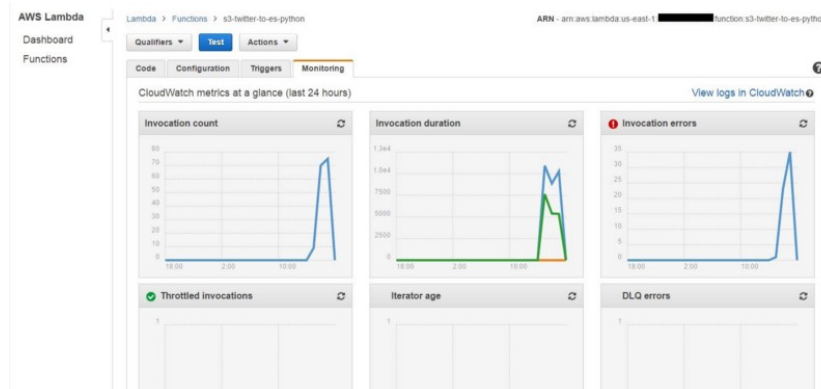
```

92 def get_tweet(doc):
93     tweet = {}
94     tweet[id_field] = doc[id_field]
95     tweet['hashtags'] = map(lambda x: x['text'], doc['entities']['hashtags'])
96     tweet['coordinates'] = doc['coordinates']
97     tweet['timestamp_ms'] = doc['timestamp_ms']
98     tweet['text'] = doc['text']
99     tweet['user'] = {'id': doc['user']['id'],
100                     'name': doc['user']['name'],
101                     'location': doc['user']['location'],
102                     'lang': doc['user']['lang'],
103                     'statuses_count': doc['user']['statuses_count'],
104                     'favourites_count': doc['user']['favourites_count'],
105                     'followers_count': doc['user']['followers_count'],
106                     'friends_count': doc['user']['friends_count']}
107
108     tweet['mentions'] = re.findall(r'@\w*', doc['text'])
109     tweet['in_reply_to_user_id_str'] = doc['in_reply_to_user_id_str']
110     tweet['in_reply_to_screen_name'] = doc['in_reply_to_screen_name']
111     tweet['retweeted'] = doc['retweeted']
112     tweet['retweet_count'] = doc['retweet_count']
113     tweet['place'] = doc['place']
114     _sentiment_analysis(tweet)
115     return tweet

```

Adding data fields

- Zip up the directory again
- Go to your function in the Lambda Management console.
- Upload the new .zip file. It will replace the old function.
- Go to the monitoring tab.
- Click on 'View logs in CloudWatch' to verify the new function works properly. These logs are very useful to debug any errors in the python script.



View console activity

Add new indices to Kibana

Kibana helps you to aggregate your data. When working with different data sets it may be useful to add another index or two to segregate your information streams. You can do it like this:

- Go to the s3-twitter-to-es-python directory.
- Edit twitter_to_es.py to update the index name.

```
11  
12  
13     #index_name = 'twitter' #ORIGINAL VALUE  
14     index_name = 'twitterv4'  
15     doc_type = 'tweet'
```

Add an new index

- Zip up the directory again
- Go to your function in the Lambda Management console.
- Upload the new .zip file. You will see the new index listed on the 'Indices' tab for for Elasticsearch domain.

Discover and analyze data

The tweet data is transferred to the S3 bucket you configured every 5 minutes or when it reaches a particular size. Depending on how much data you decided to capture, you should start to see data files appearing in S3 very soon. Once the first data item appears in S3, the Lambda function should process it.

The first processing iteration should happen in about five minutes. If you don't see any output from the lambda function, first verify that data is being written to S3. Then check that the S3 bucket permissions are correct for reading. It is ok to open the bucket permissions to be world readable since the bucket only holds publicly available data anyway. NOTE: You should choose more restrictive access permissions for more sensitive data.

When Elasticsearch confirmed your domain above, a URL was created for Kibana. Don't be disappointed if the URL didn't link to anything interesting then. Kibana won't return anything until after the Lambda function has processed the first set of data. Once that happens you are free to explore!

Happy tweet prospecting!

